Normes et guidelines de développement — FileFlow

Titre	Normes et guidelines de développement — Plateforme FileFlow
Version	1.1
Date	27 octobre 2026
Statut	Validé
Propriétaire	Driss OUAAZIZ — Software Architect

Ce document définit les conventions de code, les exigences de qualité et le processus de contribution du projet FileFlow. Il s'applique à l'ensemble du dépôt (backend Rust, frontend React) et complète le Dossier d'Architecture Technique (doc 03) et le Dossier de sécurité (doc 06).

1. Organisation du dépôt

```
fileflow/
├── backend/            Service Rust (API axum)
│   ├── src/
│   │   ├── lib.rs      Crate bibliothèque : logique, modules, point d'entrée run()
│   │   ├── main.rs     Binaire mince : appelle fileflow::run()
│   │   └── *.rs        Modules par domaine (files, users, shares, auth, …)
│   ├── fuzz/           Cibles de fuzzing (cargo-fuzz)
│   ├── Cargo.toml      Crate lib + bin, niveaux de lints clippy
│   ├── clippy.toml     Exceptions clippy propres aux tests
│   ├── deny.toml       Politique cargo-deny (licences, sources, crates bannies)
│   └── .cargo/audit.toml  Avis cargo-audit neutralisés et justifiés
├── frontend/           Application React + TypeScript (Vite)
│   ├── src/
│   │   ├── components/ Composants d'interface
│   │   ├── api.ts      Client HTTP
│   │   ├── store.ts    État applicatif (Zustand)
│   │   ├── sanitize.ts Assainissement HTML (DOMPurify)
│   │   └── …
│   └── eslint.config.js  Règles ESLint (correction et sécurité)
├── documentation/      Documentation projet (ce dossier)
├── .github/workflows/  Intégration continue (ci.yml, security.yml)
└── .github/dependabot.yml  Mises à jour automatisées des dépendances
```

Le backend est structuré en **crate bibliothèque + binaire** : toute la logique réside dans `lib.rs` (exposée en `pub mod`), et `main.rs` se limite à lancer le runtime. Cette séparation permet aux tests d'intégration et aux cibles de fuzzing de lier le code comme une bibliothèque.

2. Conventions Rust (backend)

- **Formatage** : `cargo fmt` (rustfmt) obligatoire avant tout commit ; aucune divergence de style tolérée en revue.
- **Analyse statique** : `cargo clippy --all-targets --locked -- -D warnings` doit passer : tout avertissement, y compris dans les tests, est une erreur bloquante en intégration continue. Les niveaux de lints du projet sont déclarés dans la section `[lints.clippy]` de `Cargo.toml` (`unwrap_used`, `dbg_macro`, `todo`) et non par des attributs dispersés dans le code.
- **Gestion d'erreurs** : toutes les fonctions de handler retournent `Result<T, AppError>`. `AppError` centralise la conversion en réponse HTTP (code + enveloppe `{ "error": … }`). Ne jamais propager une erreur brute vers le client.
- **Pas de `unwrap()` / `expect()` sur une donnée externe hors des tests.** Les valeurs issues de requêtes, de fichiers ou de la base (chemins, en-têtes, corps, paramètres, contenus désérialisés) sont traitées de manière faillible (`?`, `ok_or`, `unwrap_or_default`…). Le lint `clippy::unwrap_used` interdit `unwrap()` dans le code de production ; il reste autorisé dans les tests (`allow-unwrap-in-tests` dans `clippy.toml`). Un `expect()` n'est admis que sur un invariant interne prouvé, avec un message qui l'énonce, ou au démarrage pour refuser de lancer le service sans une configuration obligatoire (par exemple `ADMIN_PASSWORD` lors de l'initialisation). Le profil release utilise `panic = "abort"` : toute panique devient un arrêt de processus, donc un déni de service.
- **Modularité** : un module par domaine métier ; les fonctions pures et réutilisables (résolution de chemin, validation) sont isolées et testées unitairement.
- **Concurrence** : les accès partagés passent par les primitives `tokio` (`RwLock`, `Mutex`) ; les écritures persistées sont sérialisées. Éviter tout verrou tenu au travers d'un `await` prolongé.

---

3. Conventions TypeScript / React (frontend)

- **TypeScript strict** : `tsc --noEmit` doit passer sans erreur. Le typage explicite est préféré ; `any` est proscrit hors cas justifié et commenté.
- **Lint** : `pnpm lint` (ESLint, configuration `eslint.config.js` : règles recommandées JavaScript et `typescript-eslint`, `@eslint-react`, `eslint-plugin-security`, `eslint-plugin-no-unsanitized`) doit passer sans erreur ; il est bloquant en intégration continue. Une règle n'est désactivée que ponctuellement (`eslint-disable-next-line <règle> -- <justification>`), jamais pour un fichier entier. Les règles d'hygiène React encore en avertissement sont corrigées au fil de l'eau, puis promues en erreur.
- **Composants** : un composant par fichier, nommé en `PascalCase` ; les hooks respectent les règles de React (pas d'appel conditionnel). La logique réseau est concentrée dans `api.ts`, l'état partagé dans le store Zustand (`store.ts`).
- **Rendu HTML** : les puits XSS (`dangerouslySetInnerHTML`, affectations `innerHTML` / `outerHTML`, URL `javascript:`, `target="_blank"` sans `rel="noopener"`, `iframe` sans `sandbox` ou avec un `sandbox` dangereux) sont des erreurs ESLint. Un contenu HTML n'est injecté que s'il a été assaini par `sanitize.ts` (DOMPurify), en plus de l'échappement à la source (rendu Markdown, coloration syntaxique) ; chaque site d'injection porte un commentaire de désactivation explicite indiquant comment son entrée a été assainie. `eval`, `new Function` et équivalents sont interdits. Voir la règle de sécurité §4.
- **Expressions régulières** : les fichiers des utilisateurs sont rendus côté client ; toute expression régulière exposée à leur contenu doit être de complexité linéaire (règle `security/detect-unsafe-regex` en

erreur). Lorsque la règle signale à tort une expression, la désactivation ponctuelle renvoie au test qui mesure son comportement sur une entrée hostile (`markdown.test.ts`).

- **Dépendances client** : préférer des bibliothèques sans dépendances transitives lourdes ; toute nouvelle dépendance passe l'audit (`pnpm audit`).

---

4. Règles de sécurité du code (OWASP Secure Coding)

Ces règles sont non négociables et vérifiées en revue et par l'analyse statique.

- **Validation des entrées** : tout chemin utilisateur est normalisé et confiné à la racine des données (rejet de la traversée `..` et des liens symboliques s'échappant) ; les noms de fichiers sont assainis (séparateurs, caractères de contrôle) ; les extensions dangereuses sont bloquées au téléversement.
- **Encodage de sortie** : aucun contenu utilisateur n'est rendu en HTML actif. Côté frontend, DOMPurify assaine les sorties ; côté backend, les fichiers de contenu actif sont servis en `text/plain`. Les pages HTML générées côté serveur échappent systématiquement les valeurs interpolées.
- **Requêtes SQL** : exclusivement **paramétrées** (liaison de paramètres). Aucune interpolation de valeur utilisateur dans une requête. Les seuls identifiants interpolés (noms de tables) proviennent de constantes internes.
- **Secrets** : jamais dans le code ni les journaux. Le secret JWT est généré au premier démarrage et persisté avec des droits restreints (`0600`) ; les mots de passe sont hachés avec bcrypt ; les secrets TOTP et codes de récupération ne sont jamais renvoyés en clair après leur émission initiale.
- **Chemins virtuels** : tout chemin reçu du client est ramené à sa forme canonique par `files::normalize_virtual` **avant** tout contrôle, et c'est cette forme qui est contrôlée, accédée et persistée. Pour savoir si un chemin est couvert par un préfixe (chemin autorisé de groupe, partage, dossier collaboratif, purge ou rebasage), utiliser exclusivement `files::path_covers` : une comparaison brute `starts_with` sur des chemins est proscrite (`/equipeA` ne doit pas couvrir `/equipeA-archives`, ni `/equipeA/../equipeB` passer le contrôle). Un partage, un lien ou un favori se crée sur un chemin direct, jamais au travers d'un lien symbolique (`ensure_direct_path`).
- **Contrôle d'accès** : centralisé dans les points de contrôle de `files.rs`. Toute nouvelle route manipulant une ressource doit passer par l'un d'eux, avec la permission adéquate :
  - `validate_access` pour un accès ordinaire (partages internes et dossiers collaboratifs inclus) ;
  - `validate_own_access` lorsqu'un utilisateur **délègue** un accès (octroi d'un partage interne, création d'une demande de dépôt) : un accès reçu par partage ne se re-partage pas ;
  - `own_grant` pour borner une autorité dérivée (dossier collaboratif) aux droits propres de son créateur. L'autorité dérivée se vérifie **à chaque accès**, jamais une seule fois à la création, afin que la révocation du créateur se propage. Les points d'accès publics (liens, dépôts) revérifient les droits du créateur à chaque requête et répondent par une erreur uniforme qui ne révèle pas la cause du refus.
- **Journalisation** : les actions sensibles sont auditées ; aucune donnée secrète (mot de passe, jeton) n'est journalisée.

---

5. Tests

- **Tests unitaires backend** : suite colocalisée (`#[cfg(test)]`) couvrant les magasins de données, la logique de contrôle d'accès, la résolution de chemin, le parsing des plages HTTP et la limitation de débit — **103 tests** à ce jour, tous verts avant fusion (`cargo test --locked`).

- **Fuzzing** : les analyseurs consommant une entrée non fiable disposent de cibles `cargo-fuzz` (`backend/fuzz/`) — résolution de chemin, assainissement de nom, parsing de plage, en-tête `Content-Disposition`. Toute évolution de ces fonctions **exige** de rejouer les cibles correspondantes.
- **Couverture des fonctions sensibles** : toute fonction touchant à la sécurité (contrôle d'accès, confinement de chemin, authentification) doit être couverte par un test avant fusion.
- **Test de non-régression par correctif de sécurité** : chaque vulnérabilité corrigée est accompagnée d'au moins un test qui reproduit le scénario d'exploitation et échoue sans le correctif (contournement de préfixe, re-partage transitif, épuisement d'un plafond de téléchargements, verrouillage d'un compte par un tiers, etc.).
- **Frontend** : tests unitaires Vitest (`pnpm test`, **114 tests** à ce jour), `tsc --noEmit` et build de production (`pnpm run build`) doivent réussir. Les évolutions d'interface sont vérifiées manuellement sur les parcours impactés.

---

6. Definition of Done

Une contribution est « terminée » lorsque **toutes** les conditions sont réunies :

- ☐ Le code compile sans avertissement (`cargo build`, `tsc`).
- ☐ `cargo fmt` et `cargo clippy --all-targets -- -D warnings` passent ; `pnpm lint` passe sans erreur.
- ☐ Les tests unitaires passent (`cargo test`) ; les cibles de fuzzing concernées ont été rejouées le cas échéant.
- ☐ Les fonctions sensibles ajoutées/modifiées sont testées ; tout correctif de sécurité inclut son test de non-régression.
- ☐ Les portes de sécurité de l'intégration continue sont vertes (§8).
- ☐ La documentation impactée est mise à jour (API, DAT, ADR, registre de dette).
- ☐ Aucun secret ni donnée personnelle introduit dans le dépôt.
- ☐ La contribution a été revue et approuvée (§7).

---

7. Processus de revue de code

- Chaque évolution passe par une **branche dédiée** et une demande de fusion ; aucune poussée directe sur la branche principale.
- **Au moins une revue** approuvée est requise. Les changements touchant la sécurité, le contrôle d'accès ou la cryptographie requièrent une attention renforcée et, idéalement, un second relecteur.
- La revue vérifie : correction fonctionnelle, respect des conventions (§2–§4), couverture de test, absence de régression de sécurité, lisibilité et cohérence avec le code existant.
- Les décisions d'architecture prises en cours de revue sont consignées sous forme d'**ADR** (voir doc 05).

---

8. Intégration continue et portes qualité

Deux workflows s'exécutent à chaque poussée et demande de fusion ; leurs portes sont **toutes bloquantes**.

Le workflow `.github/workflows/ci.yml` vérifie la correction du code :

PORTE	OUTIL	PORTÉE
Lint frontend	ESLint (`pnpm lint`)	Correction React/TypeScript, puits XSS, expressions régulières vulnérables
Traductions	`pnpm run check:i18n`	Cohérence des catalogues de traduction
Tests frontend	Vitest (`pnpm test`)	Suite unitaire frontend
Build frontend	`pnpm run build`	Vérification de types et build de production
Lint backend	`cargo clippy --all-targets --locked -- -D warnings`	Code et tests Rust
Tests backend	`cargo test --locked`	Suite unitaire backend

Le workflow `.github/workflows/security.yml` exécute en outre, de façon planifiée chaque lundi à 06:00 UTC, les portes de sécurité suivantes :

PORTE	OUTIL	PORTÉE
Analyse statique (SAST)	semgrep	Code Rust + TypeScript/React
Vulnérabilités de dépendances + IaC	trivy (filesystem)	Verrous Cargo/npm/pnpm, Dockerfile
Image conteneur	trivy (image)	Couches de l'image livrée
Avis Rust	cargo-audit	Dépendances Cargo (base RustSec)
Politique de dépendances Rust	cargo-deny	Avis, licences en liste blanche, sources (crates.io uniquement), crates bannies
Avis JS/TS	pnpm audit	Dépendances frontend
Détection de secrets	gitleaks	Historique Git complet
Fuzzing (planifié)	cargo-fuzz	Analyseurs d'entrées non fiables

Le détail de la stratégie de sécurité et des résultats figure dans le Dossier de sécurité (**doc 06**). Aucune fusion n'est autorisée avec une porte au rouge.

---

9. Gestion des dépendances

- **Frontend** : gestionnaire **pnpm**, installation en **verrou figé** (`pnpm install --frozen-lockfile`). Le `pnpm-lock.yaml` fait référence ; toute mise à jour de dépendance est explicite, tracée et re-auditée.
- **Backend** : `Cargo.lock` versionné et figé pour les builds reproductibles ; les mises à jour passent par `cargo update` ciblé et re-audit.
- **Ajout d'une dépendance** : justifié (pourquoi, alternatives écartées), audité, et privilégiant les composants maintenus et à faible surface transitive.
- **Licences et provenance** : `cargo-deny` n'admet que des licences permissives en liste blanche (MIT, Apache-2.0, BSD, Zlib, Unicode-3.0…) et des crates issues de crates.io ; toute autre licence, toute dépendance Git ou de registre alternatif, toute version joker exige une décision explicite consignée dans `backend/deny.toml` . Les moteurs de compression zstd et bzip2 y sont bannis.

- **Mises à jour automatisées** : Dependabot (`.github/dependabot.yml`) couvre cargo, npm, GitHub Actions et Docker. Les mises à jour de version sont proposées chaque lundi, groupées pour les versions mineures et correctives, et seulement pour des versions publiées depuis au moins 7 jours (délai de carence) ; les correctifs de sécurité sont proposés dès la publication de l'avis. Chaque demande de fusion Dependabot passe les mêmes portes que toute contribution ; une montée de version majeure est traitée comme une évolution (lecture des notes de version, adaptation du code, tests).
- **Avis sans correctif** : un avis non corrigeable et non atteignable est documenté et neutralisé explicitement, avec la justification et la condition de levée, de façon synchronisée dans `backend/.cargo/audit.toml` (cargo-audit) et `backend/deny.toml` (cargo-deny).

Références

- **Clean Code**, Robert C. Martin — principes de lisibilité et de maintenabilité.
- **Google Style Guides** — https://google.github.io/styleguide/ — conventions de code par langage.
- **OWASP Secure Coding Practices** — https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/ — référentiel des règles de codage sécurisé.